

Lab 3 – Architectural Justification

Food Bank Surplus Redistribution Platform — Problem Statement #62

Name: MANASA K SRN: PES1UG25CS824

Architecture Selection: *“We chose Microservices Architecture for the Food Bank Surplus Redistribution System.”*

Architectural Choice

We selected Microservices Architecture, decomposing the platform into five independent, loosely-coupled services — Food Donation, Food Discovery & Reservation, Notification Service, Pickup & Expiry Lifecycle, and a shared Database — each communicating through well-defined provided/required interfaces. This mirrors the three functional epics already used in the Lab 2 backlog (Food Donation Management, Food Discovery & Reservation, and Pickup & Food Lifecycle).

Two Reasons

- Independent scaling for bursty, latency-sensitive workloads: NFR-001 requires proximity push notifications to reach all shelters within 5 km within 10 seconds during peak load of 500 concurrent postings, while NFR-002 requires core pages (post, browse, reserve) to load in under 3 seconds on a throttled 3G connection. Microservices lets the Notification Service scale independently during donation spikes without degrading the Food Discovery & Reservation service's response time.
- Fault isolation aligned with real bounded contexts: the system already separates naturally into donation intake, discovery/reservation, and pickup/expiry lifecycle (including the independent Scheduler actor for auto-expiring batches). If the Pickup & Expiry Lifecycle service fails, Donor Partners can still post batches and Shelter Coordinators can still browse listings — a food-safety-critical property a single monolithic client-server deployment would not guarantee.

Security Advantage

Each microservice authenticates and encrypts its own API boundary independently (per NFR-001's requirement for an authenticated, encrypted notification channel), so shelter-verification credentials, donor data, and pickup verification codes stay isolated within their owning service rather than one shared data layer. A compromise of one service (e.g. Notification) does not automatically expose another service's data (e.g. the Pickup & Expiry Lifecycle service's verification codes), limiting the blast radius of a breach.

Performance Benefit

Because each service is deployed and scaled independently, the Food Discovery & Reservation service — which must return filtered search results within 2 seconds (FR-002) and load under 3 seconds on 3G (NFR-002) — is never blocked by heavier, asynchronous operations such as notification fan-out or the Scheduler's periodic auto-expire sweep (FR-004). These background-heavy tasks run in their own services with their own resources, keeping the user-facing read/reserve paths fast even during large notification bursts.